Failure-driven parameter repair

The equations behind the Kimi implementation

Code-grounded mathematical companion November 3, 2026

Purpose. Translate the implemented solvers in KimiBrainSurgery into explicit objectives, constraints, and parameter updates. This document is an analysis of saved code and results, not a newly executed repair experiment. It provides a common mathematical interface for eventually combining Kimi's analytic solvers with the separate LLM-proposed repair in FailureIsAllYouNeed.

Source snapshot. All implementation claims refer to commit

`13cfaec9f484a1a8864349a814bfd62905022a05`.

Run records were checked through the W&B MCP connection against the Kimi Brain Surgery project: 18 sweep runs, one monotonic run, and one paired-evaluation run. The repository includes copied numerical records and their source hashes.

The central operation. Given a failed decision, choose an update that increases a saving action's logit relative to the wrong action while penalizing changes on other observations. The basic subproblem is

$$\min_{\delta} \frac{1}{2}\delta^{\mathsf{T}} M \delta \quad \text{subject to} \quad a^{\mathsf{T}}\delta = t, \qquad \delta^{*} = \frac{t\, M^{-1}a}{a^{\mathsf{T}} M^{-1} a}. \tag{1}$$

Here a describes sensitivity of the target decision, M penalizes edit size and interference, and t is the required logit-gap correction. The formula requires $M \succ 0$ and $a \neq 0$.

What is exact, and what is local?

Method	Mathematical status
Readout W_3	Hidden activations stay fixed. The selected pair's logit-gap change is exact.
Rank-one W_2	Exact preactivation change followed by a first-order approximation through tanh.
Unrestricted W_2	The same local approximation with all 32×32 entries available, solved in blocks.
Keep/rollback	Nondecreasing mean return on the reused 64-game gate, by construction. No global guarantee.

Interpretation. This is constrained inverse behavior design. It is not an inverse of the original training trajectory. The W_2 solvers explicitly use analytic derivatives; they do not run a gradient-descent training loop. Kimi's language-model weights are not being modified. The edited object is a small, previously PPO-trained game policy.

1 Policy, failure signal, and diagnosis

Use column vectors; the source uses equivalent row-vector NumPy expressions. For a flattened one-hot grid observation $x \in \mathbb{R}^{500}$,

$$\begin{aligned} u(x) &= \tanh(W_1^\mathsf{T} x + b_1) \in \mathbb{R}^{32}, \\ z(x) &= W_2^\mathsf{T} u(x) + b_2, \qquad h(x) = \tanh z(x) \in \mathbb{R}^{32}, \\ \ell(x) &= W_3^\mathsf{T} h(x) + b_3 \in \mathbb{R}^5, \qquad \pi(x) = \arg\max_c \ell_c(x). \end{aligned} \tag{2}$$

Shapes are $W_1 : 500 \times 32$, $W_2 : 32 \times 32$, $W_3 : 32 \times 5$; bias lengths are 32, 32, and 5. Total: 17,253 parameters. Actions 0, 1, and 2 leave the paddle still; 3 moves left and 4 moves right.

From a trajectory to an avoidable decision

The miner predicts the next ball contact column c_* and the number T of remaining ticks. For current paddle center p, let $p_c = \mathrm{clip}(p + \mathrm{move}(c), 2, 7)$. After that action, the paddle centers reachable before contact form

$$I_c = [\max(2, p_c - (T-1)),\ \min(7, p_c + (T-1))]. \tag{3}$$

A width-three paddle can still intercept the ball if $I_c \cap [c_* - 1, c_* + 1] \neq \varnothing$. The saving set S_f contains the canonical actions $\{0, 3, 4\}$ satisfying this test. A failure is recorded when the chosen action destroys this reachability and S_f is nonempty. A saving action preserves an available route to contact; it does not guarantee that the edited policy will choose that route later.

The original miner considers descending balls only. Its landing predictor is a simplified descent calculation and aborts on a predicted diagonal brick hit. The later `use_flight=True` path simulates ascent, wall and brick collisions, and brick removals until the next paddle-row arrival, with a 1,000-tick cap. It mirrors the simulator's ball physics. This exposes failures to pre-position the paddle during ascent that the original miner cannot see.

Default mining uses seeds 0–199 and at most 200 steps per game. Every visited observation becomes a probe; failures are deduplicated by observation bytes. The outer loop selects the first eligible failure.

Which logit contributions favor the wrong action?

At failure x_f, set $a = \pi(x_f)$ and $b = \arg\max_{c \in S_f} \ell_c(x_f)$. The initial wrong-minus-saving gap is

$$g = \ell_a(x_f) - \ell_b(x_f) = \sum_{j=1}^{32} h_{f,j}(W_{3,ja} - W_{3,jb}) + (b_{3,a} - b_{3,b}). \tag{4}$$

The diagnostic sorts the hidden-unit summands and reports the three largest. The bias term is part of the exact gap but is absent from that ranked list. This decomposition explains the current logits; it does not identify a unique causal parameter subset or imply that only three units will be edited.

Source: `failure_gym.py`: `mine`, `flight_landing`, `diagnose`; `assets/policy.py`: forward pass and action selection.

2 Exact readout edits: naive and probe-protected W_3

Let $m > 0$ be the desired pairwise margin ($m = 0.1$ by default) and $t = g + m$. Only two output columns change:

$$\Delta W_3[:, a] = -\tfrac{1}{2}d, \qquad \Delta W_3[:, b] = +\tfrac{1}{2}d, \qquad \Delta b_3 = 0. \tag{5}$$

Since h_f is unchanged, $g_{\text{new}} = g - h_f^\top d$ exactly. Enforcing $h_f^\top d = t$ therefore gives $\ell_b^{\text{new}} - \ell_a^{\text{new}} = m$. Other action logits can still exceed ℓ_b^{new}: the gate separately checks whether the global argmax belongs to S_f.

The minimum-size update without probe protection is

$$\min_d \|d\|_2^2 \text{ s.t. } h_f^\top d = t \quad \Longrightarrow \quad d = \frac{t}{\|h_f\|_2^2} h_f. \tag{6}$$

It changes up to 64 scalar entries, not one isolated weight.

A soft preservation penalty

Stack current probe hidden activations as rows of $H \in \mathbb{R}^{N \times 32}$. The implemented protected edit solves

$$\begin{aligned} &\min_d \quad \|d\|_2^2 + \frac{\lambda}{N}\|Hd\|_2^2, && \text{s.t.} \quad h_f^\top d = t, \\ &M = I + \frac{\lambda}{N} H^\top H, && d^* = \frac{t\, M^{-1} h_f}{h_f^\top M^{-1} h_f}. \end{aligned} \tag{7}$$

Under the symmetric update, $\|\Delta W_3\|_F^2 = \|d\|_2^2/2$ and the total squared movement of the two affected probe logits is $\|Hd\|_2^2/2$. The objective therefore balances parameter size and exact probe-logit interference, up to a common factor.

Derivation of the common solver

For Equation (1), write $L(\delta, \eta) = \frac{1}{2}\delta^\top M \delta - \eta(a^\top \delta - t)$. Stationarity gives $M\delta = \eta a$; substitution into the constraint gives $\eta = t/(a^\top M^{-1} a)$. The implementation uses a linear solve, not an explicit matrix inverse.

What "null-space" means here

At finite λ, Equation (7) is a soft quadratic penalty, not the hard condition $Hd = 0$. Larger λ favors directions that disturb the sampled probes less, but can increase edit size. Exact preservation and a nonzero target are jointly possible only if h_f has a nonzero component in $\ker H$. If H has full column rank, there is no nonzero exact null-space edit.

The probe subsampling stride is $\max(1, \lfloor P/4000 \rfloor)$ for P probes; 4,000 is a target, not a strict sample cap. N in the equations is the actual number selected. Probes are regenerated from each current policy.

Source: `failure_gym.py`: `solve`; `nullspace_gym.py`: `solve_constrained`.

3 Rank-one W_2 repair through a nonlinearity

Keep W_1, W_3, and all biases fixed. Restrict the update to

$$\Delta W_2 = u_f v^\mathsf{T}, \qquad v \in \mathbb{R}^{32}. \tag{8}$$

For any probe p, its preactivation change is exact:

$$\Delta z_p = (\Delta W_2)^\mathsf{T} u_p = (u_p^\mathsf{T} u_f) v. \tag{9}$$

Define $D_p = \mathrm{diag}(1 - h_p \odot h_p)$. The hidden change has the local expansion

$$\Delta h_p = D_p \Delta z_p + O(\|\Delta z_p\|^2). \tag{10}$$

Let $w = W_3[:, b] - W_3[:, a]$ be the saving-minus-wrong readout direction. At the failure, the linearized improvement in that logit difference is

$$w^\mathsf{T} \Delta h_f \;\simeq\; \|u_f\|_2^2 \, w^\mathsf{T} D_f v = a_f^\mathsf{T} v, \qquad a_f = \|u_f\|_2^2 \, D_f w. \tag{11}$$

Thus the local target is $a_f^\mathsf{T} v = t = g + m$.

Implemented objective and diagonal solve

With $c_p = (u_p^\mathsf{T} u_f)^2$, the subproblem is

$$\min_v \; \mu \|v\|_2^2 + \frac{\lambda}{N} \sum_{p=1}^{N} c_p \|D_p v\|_2^2 \quad \text{s.t.} \quad a_f^\mathsf{T} v = t. \tag{12}$$

Its metric is diagonal:

$$\begin{aligned} A_j &= \mu + \frac{\lambda}{N} \sum_{p=1}^{N} (u_p^\mathsf{T} u_f)^2 (1 - h_{p,j}^2)^2, \\ v_j^* &= \frac{t \, (a_{f,j}/A_j)}{\sum_k a_{f,k}^2 / A_k}, \qquad \Delta W_2 = u_f (v^*)^\mathsf{T}. \end{aligned} \tag{13}$$

The source sets $\mu = 1$. Its regularizer is $\|v\|^2$, whereas $\|\Delta W_2\|_F^2 = \|u_f\|^2 \, \|v\|^2$. The probe term protects first-order hidden activations, not directly return or even all logit margins. Small activation movement can still flip a close argmax decision.

Sequential local corrections

The code takes at most eight inner steps. It recomputes the actual failure logits, g, h_f, and D_f each step. It stops as soon as the global greedy action is in S_f; this stopping rule need not attain margin m. The probe activations and derivatives are computed once before the inner loop and stay frozen during its corrections. The initial a and b action indices also stay fixed. u_f stays fixed because W_1 is unchanged.

Each proposed step is rescaled to Frobenius norm at most 1 before application. After clipping, even its linearized equality need no longer hold. The source calls these "Newton" steps; they are sequential local gap corrections, not an exact Newton method for the full nonlinear return objective.

Source: w2_gym.py: `solve_w2`, `probe_stats`.

4 Unrestricted W_2: a block-diagonal quadratic solve

Now allow any $\Delta \in \mathbb{R}^{32\times 32}$, with columns δ_j. "Full-rank" names this unrestricted parameterization; the computed update need not have matrix rank 32. For a probe, $\Delta z_p = \Delta^\mathsf{T} u_p$ exactly and $\Delta h_p \simeq D_p \Delta^\mathsf{T} u_p$ locally. The implementation minimizes

$$\mathcal{C}(\Delta) = \mu \left\|\Delta\right\|_F^2 + \frac{\lambda}{N} \sum_p \left\| D_p \Delta^\mathsf{T} u_p \right\|_2^2. \tag{14}$$

Because the hidden-activation norm sums over units without cross-unit terms,

$$G_j = \frac{1}{N} \sum_p (1 - h_{p,j}^2)^2 u_p u_p^\mathsf{T},$$

$$\mathcal{C}(\Delta) = \sum_{j=1}^{32} \delta_j^\mathsf{T} M_j \delta_j, \qquad M_j = \mu I + \lambda G_j. \tag{15}$$

The saving-minus-wrong constraint becomes

$$\sum_j a_j^\mathsf{T} \delta_j = t, \qquad a_j = (D_f w)_j u_f. \tag{16}$$

Stacking columns would produce a 1,024-dimensional vector and a block-diagonal metric. Instead, solve 32 small systems:

$$y_j = M_j^{-1} a_j, \qquad s = \sum_k a_k^\mathsf{T} y_k, \qquad \boxed{\delta_j^* = \frac{t}{s} y_j}. \tag{17}$$

This is Equation (1) applied to the stacked vector. It avoids a dense 1024×1024 inverse. The block structure is a consequence of the chosen hidden-state penalty; a full logit-preservation penalty can introduce couplings between blocks through W_3.

Exact implementation order

1. Select a strided subset of the current probes. Compute u_p, D_p, G_j once.
2. At the current failure state, recompute g, D_f and the actual greedy action. Stop if that action belongs to S_f.
3. Set $t = g + 0.1$, form a_j, and solve Equation (17) with $\mu = 1$.
4. Rescale $\Delta \leftarrow \Delta \min(1, 1/\left\|\Delta\right\|_F)$ if nonzero, then apply $W_2 \leftarrow W_2 + \Delta$.
5. Repeat steps 2–4 up to eight corrections. Recompute the probe geometry only when the next outer repair starts.

Feasibility and numerical limits. For $\lambda \geq 0$ and $\mu > 0$, each M_j is positive definite. The denominator s is positive only if the constraint sensitivity is nonzero. Saturation, a zero feature vector, or equal readout columns can make the local target infeasible. The pinned solvers do not explicitly guard all zero-denominator cases. Float64 local diagnostics and float32 rollout evaluation also need not make identical near-tie decisions.

Source: `fullrank_gym.py`: `solve_fullrank`; `chase_unresolved.py`: same structure with configurable margin.

5 Measurement, composition, and the keep/rollback rule

For a fixed gate set G of 64 seeds (10000–10063), let

$$\widehat{J}_G(\theta) = \frac{1}{|G|} \sum_{s \in G} R(\theta, s). \tag{18}$$

Rollouts use greedy actions, float32 weights, and a 200-step horizon. The reported probe-flip fraction is

$$F_P(\theta, \theta') = \frac{1}{|P|} \sum_{x \in P} \mathbf{1}\{\pi_\theta(x) \neq \pi_{\theta'}(x)\}. \tag{19}$$

The `fixed` field checks only whether $\pi_{\theta'}(x_f) \in S_f$ at the stored observation. It does not replay the full failed trajectory or establish that the edited policy still visits that observation.

The 18 sweep runs are not monotonic loops

Each of the three solver families sweeps $\lambda \in \{0, 1, 10, 100, 1000, 10000\}$ for up to eight outer repairs. The sweep code assigns `weights = new_weights` unconditionally after each measurement, including failed local fixes and return regressions. Its "gate" is a measurement battery, not a promotion filter. These curves reveal edit interference and composition; they are not sequences of accepted improvements.

The separate monotonic loop

For a candidate $\widetilde{\theta}_k$, the actual implemented update is

$$\theta_{k+1} = \begin{cases} \widetilde{\theta}_k, & \widehat{J}_G(\widetilde{\theta}_k) \geq \widehat{J}_G(\theta_k), \\ \theta_k, & \text{otherwise.} \end{cases} \tag{20}$$

Consequently $\widehat{J}_G(\theta_{k+1}) \geq \widehat{J}_G(\theta_k)$. This is monotonicity of a finite, repeatedly consulted mean. It does not imply monotonic return on each gate seed or on any unseen distribution.

The rule does *not* additionally require `fixed=True`, a maximum probe-flip rate, or zero regressions on previously repaired observations. Those observations are retested after kept edits, but their regressions are reported rather than vetoed. Rejected failure observations are marked unresolved and excluded from later mining in the same run. The default attempt cap is 24, with an early stop after eight consecutive rollbacks. A stall or exhausted eligible mine is not proof of exhausted model capacity. The best kept gate-return checkpoint is saved in float32.

Plotting the logged run correctly. The W&B logger records the candidate's `return_after` as `mean_return` even after a rollback. The true incumbent curve must therefore use `return_after` for kept edits and `return_before` for rejected ones. The companion figure explicitly distinguishes candidate scores from the retained policy.

Targeted chase. The later seed-12 search tried W_2 and W_3 across three λ values and four margins (24 configurations). Its acceptance condition did require both a local fix and nondecreasing gate mean. Choosing among these configurations is further gate-driven selection.

Source: `monotonic_gym.py`: `run`; `wandb_log.py`: sweep and monotonic logging.

6 Saved results and what they establish

The following values are recomputed from the copied per-seed records. They are not new rollout measurements. The v1 and v3 comparisons reuse seeds 20000–20999; v4's fresh comparison uses 30000–30999.

Policy	Baseline	Repaired	Gain	Wins	Losses	Ties
v1	8.694	9.769	+1.075	137	0	863
v3	8.694	10.528	+1.834	137	17	846
v4 fresh	8.886	12.185	+3.299	253	20	727

For each paired seed s, define $d_s = R(\theta', s) - R(\theta, s)$. The mean gain and its paired standard error are

$$\bar{d} = \frac{1}{n}\sum_s d_s, \qquad \mathrm{SE}(\bar{d}) = \frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_s (d_s - \bar{d})^2}. \tag{21}$$

The v1 script reports a percentile bootstrap interval by resampling the paired differences 100,000 times with random seed 42. Pairing must be preserved; an independent bootstrap of baseline and repaired returns answers a different question. The saved v1 interval is [0.851, 1.313].

With W positive, L negative, and T tied differences, its two-sided sign test excludes ties. For $n_* = W + L > 0$, the conventional bounded expression is

$$p = \min\left(1,\ 2\sum_{i=\max(W,L)}^{n_*} \binom{n_*}{i} 2^{-n_*}\right). \tag{22}$$

The source omits the outer cap at 1; this does not affect its v1 result. For v1, $W = 137, L = 0$, giving $p = 2^{1-137} \simeq 1.15 \times 10^{-41}$. This tests sign symmetry among non-ties, not model-wide safety or absence of gate overfitting.

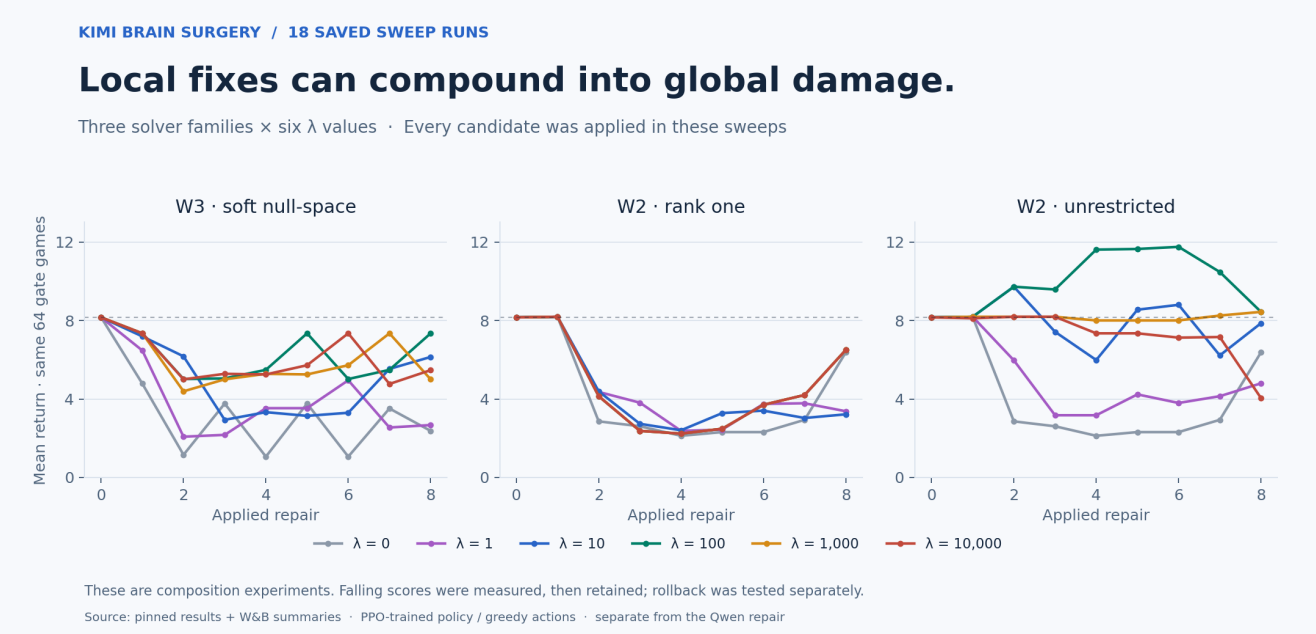
Zero observed v1 losses have a finite scope. The 1,000-seed evaluation contains zero regressions. It does not prove that none exist elsewhere. The first 64 seeds were already reported as a pilot; the larger evaluation includes them and is not an independent second sample. Its value as a held-out check depends on the checkpoint remaining fixed after that pilot.

The v3 regression is informative. Seventeen old-set seeds lose 13 points each. Source analysis attributes this class to failure to pre-position the paddle during ascent, outside the original descent-only miner's coverage. The flight miner makes such states detectable. The v4 fresh evaluation still contains 20 losses, despite higher average return. Detection and inexpensive correction are distinct requirements.

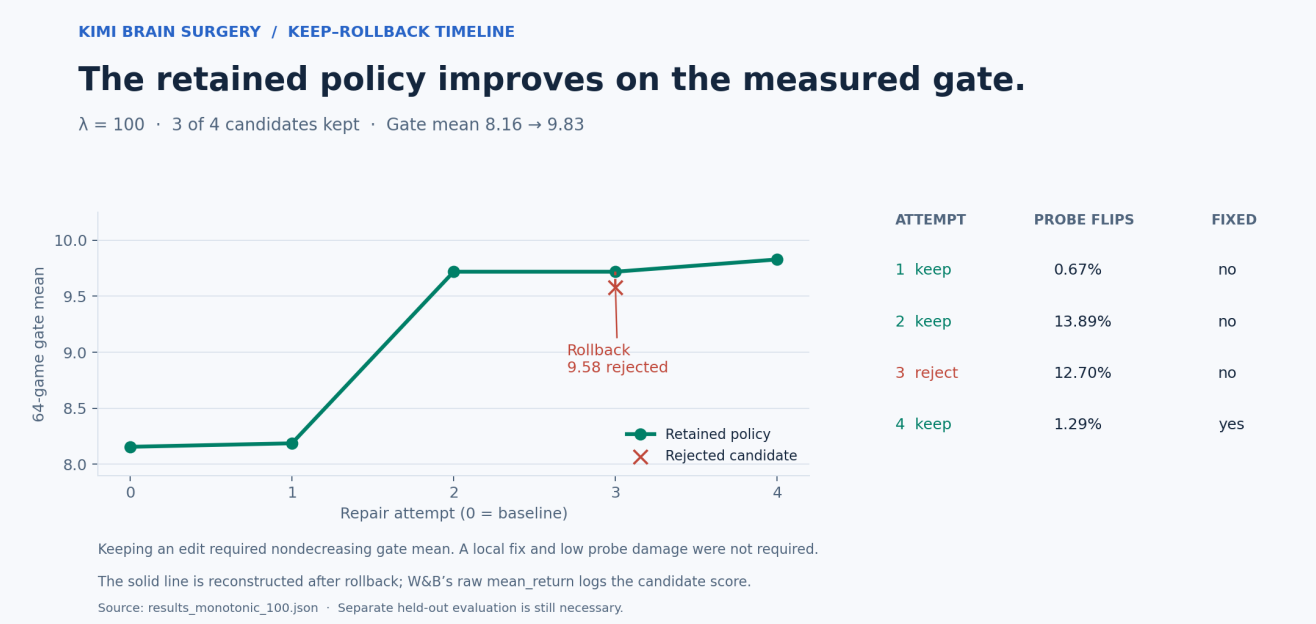
Once old-set failures guided subsequent repairs, that set became development evidence. The v4 reevaluation of all 1,000 old seeds is labeled contaminated because 17 failures informed development; it is not fresh validation. These observations support improvement on the tested distribution, with measurable remaining regressions; they do not rule out adaptive gate overfitting or establish language-model repair.

Source: `paired_eval.py`: paired scores, bootstrap, sign test; `eval/paired_heldout.json`: v1; `eval/paired_heldout_v3.json`: v3; `eval/paired_v4_fresh.json`: v4 fresh.

7 Empirical composition curves



Each line starts at the same saved baseline gate mean and then follows eight unconditionally applied repairs. A larger λ changes the local preservation metric; it does not guarantee a better trajectory-level outcome. The axis is repair number, not training steps. There are no error bars because these are exact recorded means on the same deterministic 64-game gate, not independent replicate experiments.



The solid incumbent line follows Equation (20); rejected candidate scores remain visible. Probe damage and local repair status are separate observations, not additional acceptance conditions in this run.

Plot inputs: the 18 `results_*_<lambda>.json` records and `results_monotonic_100.json`. W&B run identifiers and exact file hashes are recorded in `results/kimi/provenance.json`.

8 A precise interface for combining the two projects

Implemented here so far: two separate proposal mechanisms. KimiBrainSurgery uses analytic constrained solvers on a PPO-trained, greedily evaluated policy. The completed Qwen trial in FailureIsAllYouNeed uses tool-assisted scalar assignments on a handwritten policy with a planted reversed-controls fault and sampled actions. Its three-parameter patch improved average return on a separate gate and audit, but did not prevent the diagnostic miss. Their scores should not be pooled or directly compared as a leaderboard.

Proposed combination, not yet demonstrated. Use an editor to examine failure traces and propose a target action, layer, margin, and protected probe set; call a numerical solver to produce an explicit candidate; then use one independent verifier to test both the intended counterfactual and preservation. Log the candidate and incumbent separately, preserve exact checkpoint hashes, and deploy only accepted weights.

For a general differentiable policy, if $q(\theta)$ is a target logit difference, the local equation is

$$q(\theta + \delta) \simeq q(\theta) + J_f \delta, \qquad J_f = \nabla_\theta q(\theta)^\mathsf{T}. \tag{23}$$

A prototype extension with several constraints is

$$\min_\delta \ \tfrac{1}{2}\delta^\mathsf{T} M \delta \quad \text{s.t.} \quad A\delta = r, \qquad \delta^* = M^{-1}A^\mathsf{T}(AM^{-1}A^\mathsf{T})^{-1}r, \tag{24}$$

when $M \succ 0$ and the rows of A are independent. A pseudoinverse can handle redundancy only with the appropriate feasibility checks. Inconsistent target constraints require relaxing the problem. This multi-state formula is an extension derived here, not an implemented result in the pinned Kimi code.

This explains the useful part of "working backward": start from a desired change in behavior and solve for parameter changes that could realize it under a specified preservation metric. The sensitivity map is usually many-to-one, so this is not a unique inverse derivative. Nonlinear effects, changed future state visitation, incomplete probes, and verifier blind spots remain.

What a combined acceptance rule would need to say

A future protocol can require (i) rescue of a frozen counterfactual replay, (ii) no unacceptable regressions on a protected suite, and (iii) a predefined paired-performance criterion on reserved seeds. Those conditions must be specified before selection; a reused practice gate is not an untouched audit. An LLM's explanation of a repair is a hypothesis, not the acceptance test.

Attribution

KimiBrainSurgery is HarleyCoops' parallel work built with Kimi. This methods note translates that implementation; it does not attribute its numerical results to the separate Qwen experiment. Both sit within a conceptual replication effort inspired by Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab's Sparks of Weight-Space Programming. See the original experiment logs.

A Reproducibility and source map

Source file	Role in this document
`failure_gym.py`	Network forward pass, landing predictors, reachability, diagnosis, naive readout edit.
`nullspace_gym.py`	Exact W_3 quadratic objective and symmetric column update.
`w2_gym.py`	Rank-one parameterization, diagonal metric, local corrections, and rollout gate.
`fullrank_gym.py`	Per-unit Gram matrices and block solves for unrestricted W_2.
`monotonic_gym.py`	Mean-return keep/rollback, unresolved failures, regression reports, best checkpoint.
`chase_unresolved.py`	Targeted layer/λ/margin grid.
`paired_eval.py`	Per-seed evaluation, paired bootstrap, and sign test.
`wandb_log.py`	Sweep curves, candidate metrics, checkpoint artifacts, paired tables.

All paths above resolve at the immutable source commit on page 1. The copied results under `results/kimi/` are byte-identical to that snapshot. `provenance.json` contains their SHA-256 hashes, checkpoint hashes, source links, and a bounded W&B run snapshot. Checkpoints remain in the original project; this methods companion does not silently substitute or republish Kimi's model artifacts.

Rebuild the figures and numerical table from the root of FailureIsAllYouNeed:

```
python -m pip install -r requirements-figures.txt
python scripts/render_figures.py
python scripts/render_kimi_figures.py
python scripts/check_kimi_math.py
```

The algebra checks exercise exact W_3 gap correction, diagonal and block constraint solves, and the first-order tanh approximation. They do not rerun the original repair experiment or certify unseen-game performance.

Compile this document with a standard LaTeX installation:

```
cd docs
pdflatex -interaction=nonstopmode -halt-on-error \
  kimi-parameter-repair.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  kimi-parameter-repair.tex
```

The second pass resolves equation and page references. Figures are rendered from the saved per-attempt and per-seed data; generated plots do not smooth, invent, or interpolate unseen evaluations.

Evidence boundaries. Logs and checkpoint hashes establish what was saved. Source review establishes the implemented equations and selection rule. Fresh paired rollouts establish measured behavior for a frozen candidate on a particular distribution. None of these alone proves a unique causal mechanism, universal non-regression, indefinite improvement, or transfer to Cree or Dakota.